

Object-Oriented Programming in Java

Emily Lucia Antosch

18.09.2026

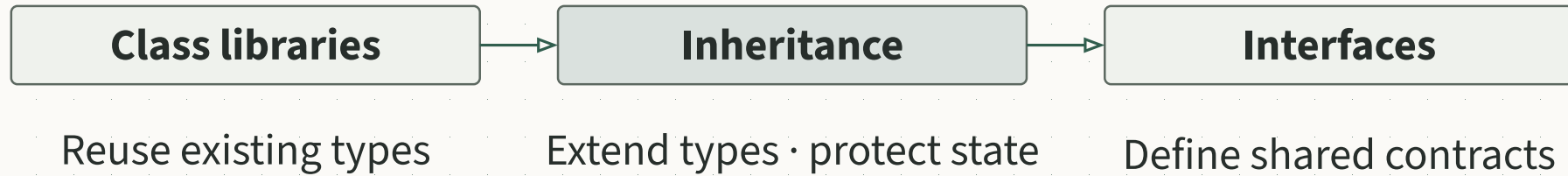
Today's route

1. Introduction	2
2. Inheritance	4
3. Object Instantiation	24
4. Referencing via Base Class	33
5. Overriding	38
6. Polymorphism	55
7. Comparing Objects	57
8. License Notice	63

1. Introduction

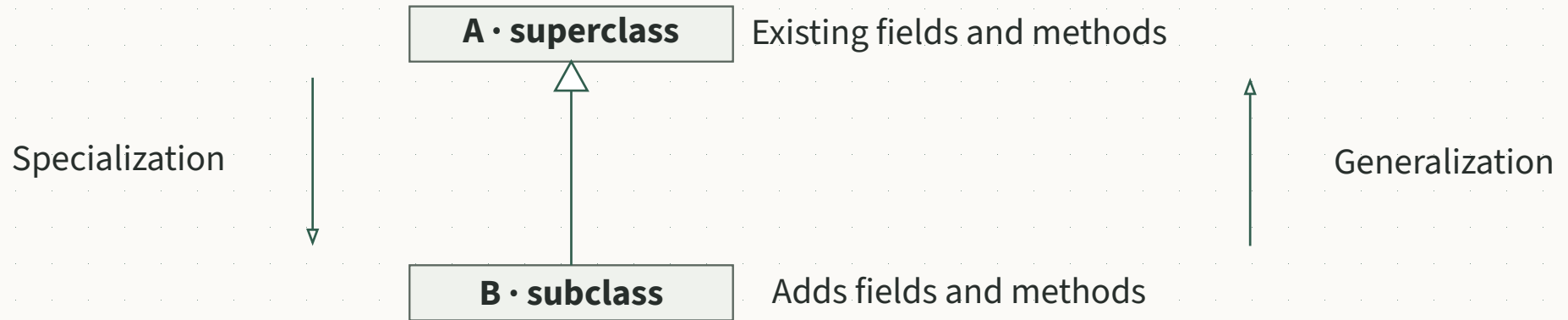
From Reuse to Inheritance

Today



2. Inheritance

Extending a Class



class B extends A • B is an A

Extending a Class

```
1  class A {  
2      // Existing fields and methods  
3  }  
4  
5  class B extends A {  
6      // Additional fields and methods  
7  }
```

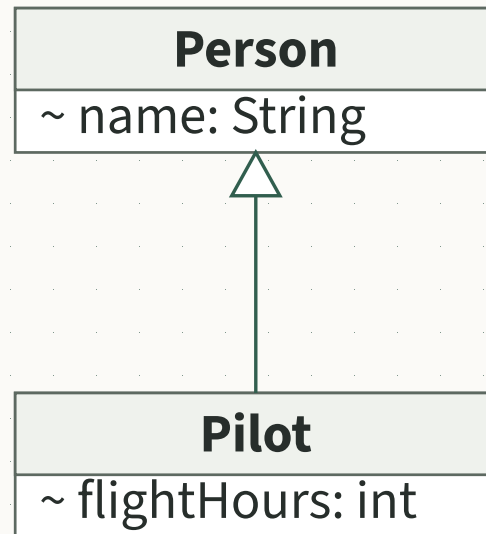
Java

Task: Person and Pilot



Task 1

Create Person and Pilot. Create a Pilot object and print its name.



Inherited: name

Added: flightHours

Task: Person and Pilot

Each public class belongs in its own file.

```
1 public class Person {  
2     String name;  
3 }  
4  
5 public class Pilot extends Person {  
6     int flightHours;  
7 }
```

Java

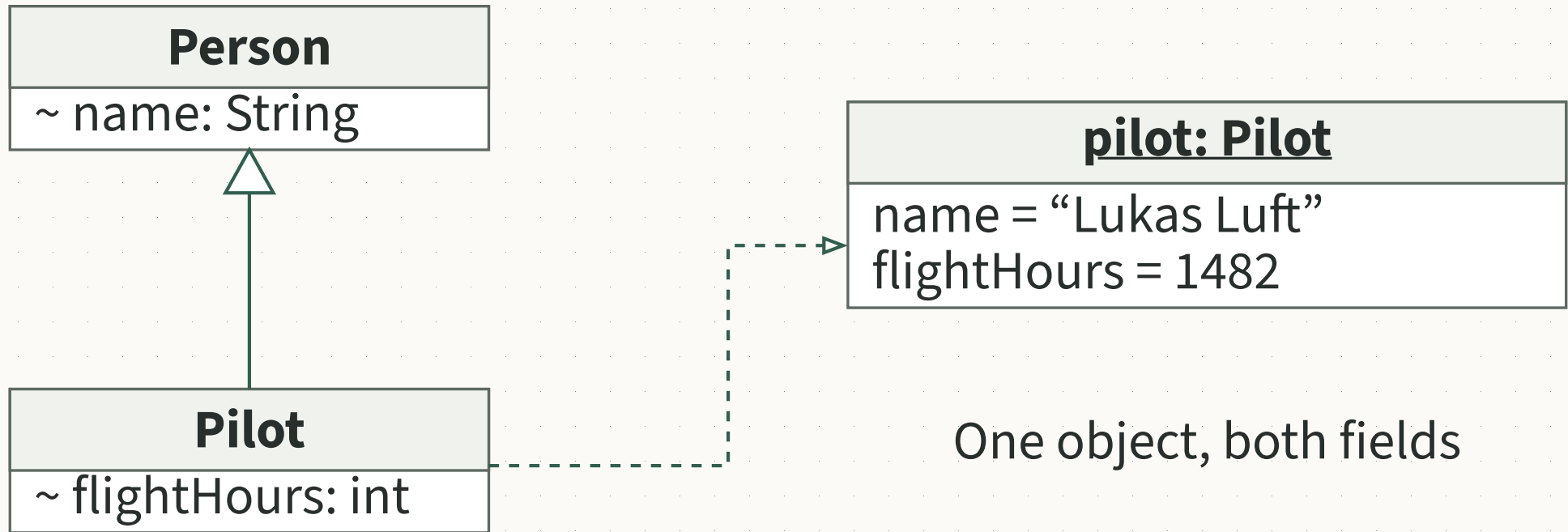
These fields have package access. Put the demo in the same package.

Task: Person and Pilot

```
1 public class PilotDemo {  
2     public static void main(String[] args) {  
3         Pilot pilot = new Pilot();  
4         pilot.name = "Lukas Luft";  
5         pilot.flightHours = 1482;  
6         System.out.println("Name: " + pilot.name);  
7     }  
8 }
```

Java

Task: Person and Pilot



One object, both fields

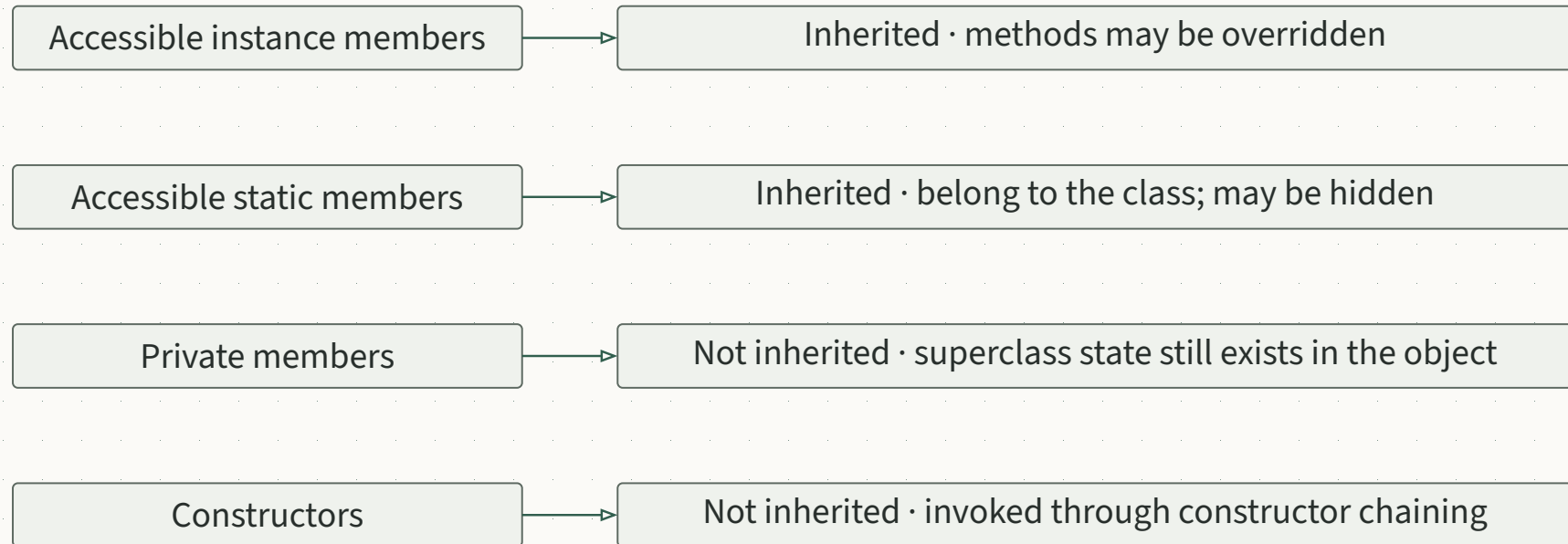
Visibility and Encapsulation

Member modifier	Same class	Same package	Subclass elsewhere	Unrelated class
public · +	yes	yes	yes	yes
protected · #	yes	yes	restricted*	no
none · ~	yes	yes	no	no
private · -	yes	no	no	no

*Outside the package: instance access through subclass-typed receivers only.

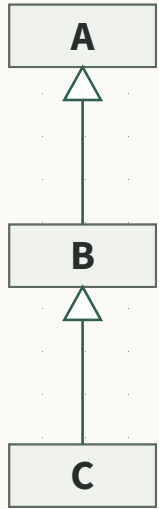
Top-level classes: public or package access. Constructors may be private.

What Is Inherited?

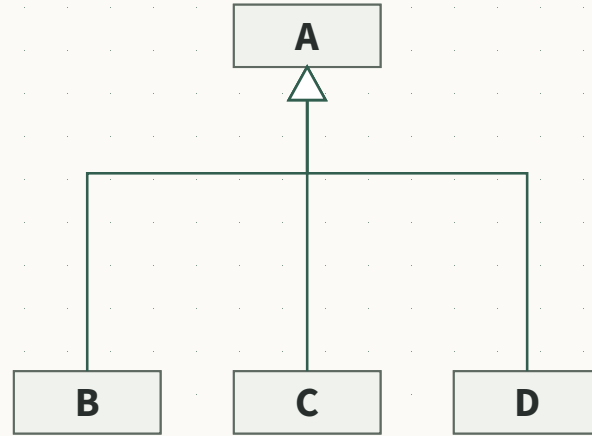


Java has no C++-style destructors.

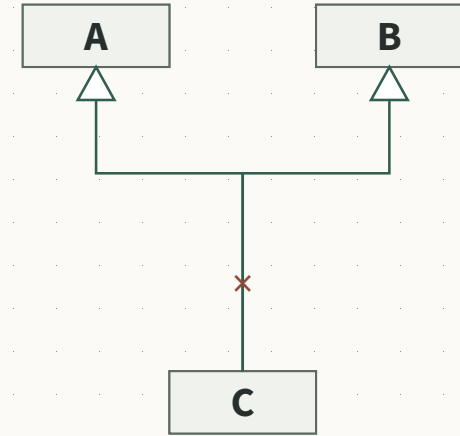
Inheritance Structures



Valid: chain



Valid: siblings



Invalid: two direct superclasses

Object: The Root Class



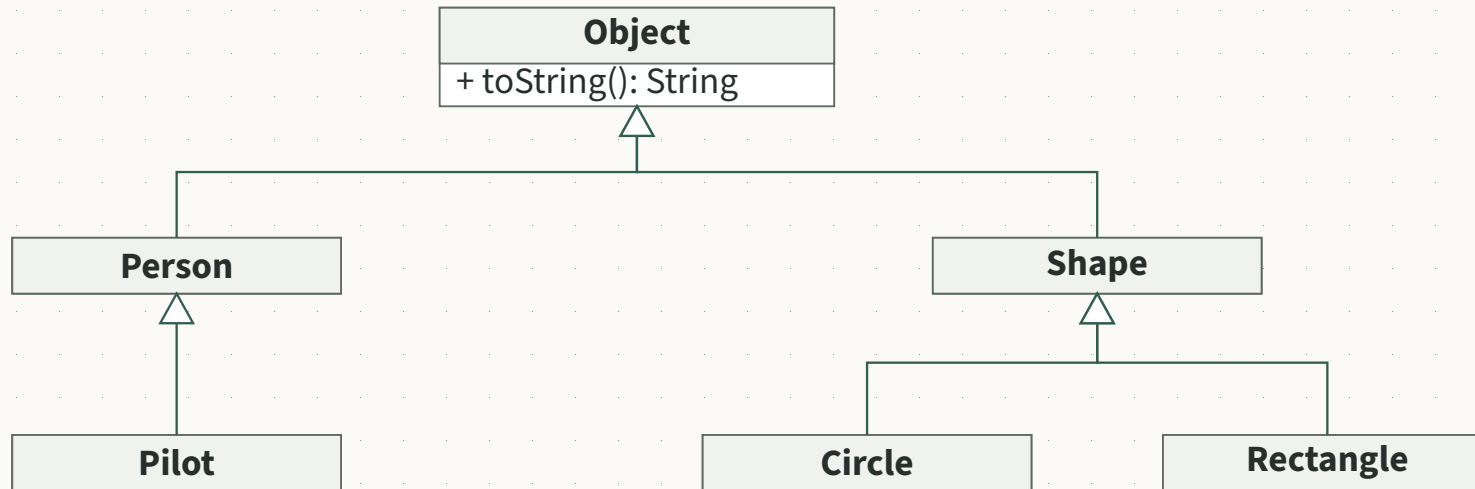
Which superclass does Person have?

```
1 public class Person {  
2     String name;  
3 }
```

Java

With no explicit extends, a class implicitly extends `Object`.

Object: The Root Class



Only Object has no superclass; every other class has exactly one direct superclass.

Object: The Root Class

Every class inherits instance methods from `Object`, including `toString()`.

```
1 Person person = new Person();  
2 person.name = "Lukas Luft";  
3 System.out.println(person.toString());
```

Java

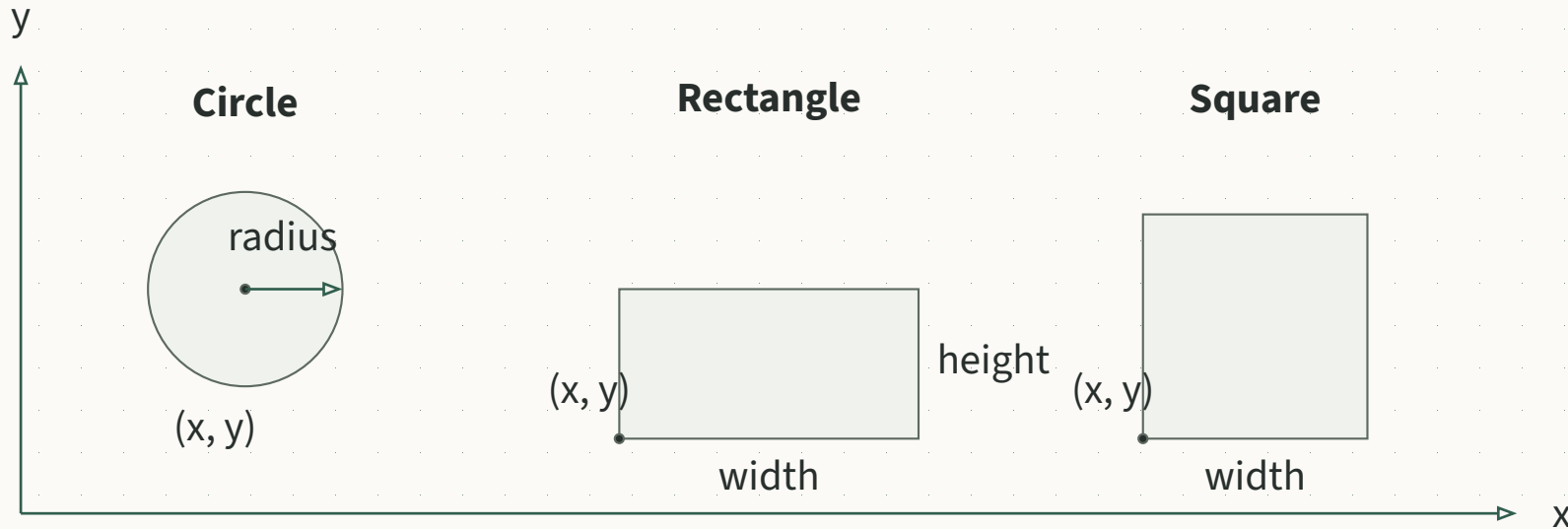
Default output: a class name, @, and a hash code in hexadecimal—not the name field.

Task: Geometric Shapes



Task 2

Model Circle, Rectangle, and Square with public fields. No methods yet.



Task: Geometric Shapes

Circle
+ x, y: double + radius: double

Same position fields

Rectangle
+ x, y: double + width, height: double

Same position fields

Square
+ x, y: double + width: double

Same position fields

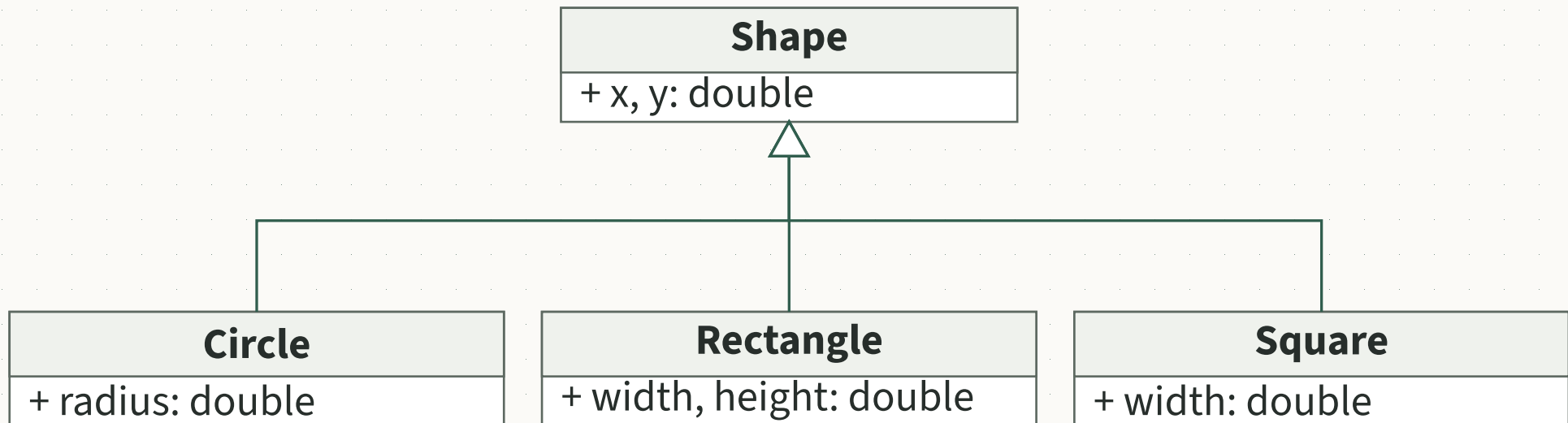
Move x and y into one common superclass.



Task 3

Move the common fields into a Shape superclass.

Task: Geometric Shapes



```
1 public class Shape {
2     public double x, y;
3 }
```

Java

Task: Geometric Shapes

```
1 public class Circle extends Shape {  
2     public double radius;  
3 }  
4 public class Rectangle extends Shape {  
5     public double width, height;  
6 }  
7 public class Square extends Shape {  
8     public double width;  
9 }
```

Java



Task 4

Add a Circle constructor that sets x, y, and radius.

Initializing Inherited Fields

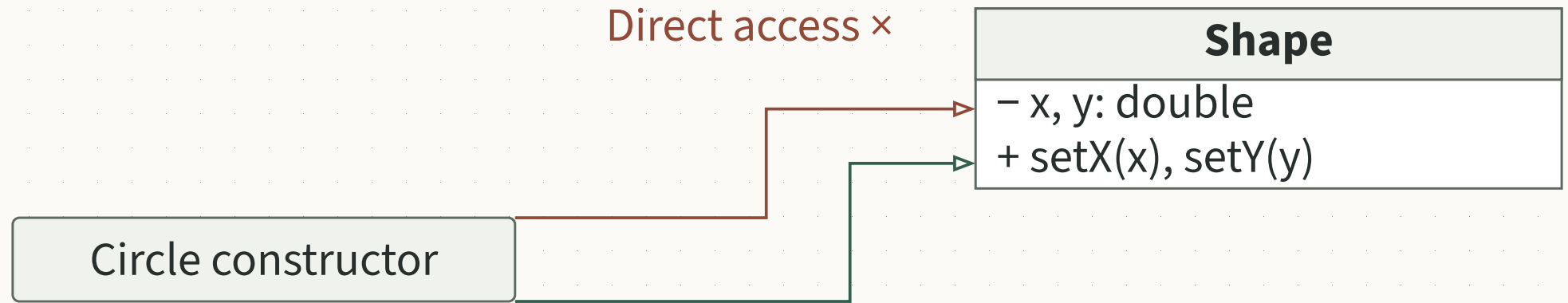
```
1 public class Circle extends Shape {  
2     public double radius;  
3  
4     public Circle(double x, double y, double radius) {  
5         this.x = x;  
6         this.y = y;  
7         this.radius = radius;  
8     }  
9 }
```

Java



What breaks if Shape makes x and y private?

Initializing Inherited Fields



Use accessible methods

Private fields still exist in the object, but Circle cannot access them directly.

Initializing Inherited Fields

```
1  public class Shape {
2      private double x, y;
3
4      public void setX(double x) {
5          this.x = x;
6      }
7      public void setY(double y) {
8          this.y = y;
9      }
10 }
```

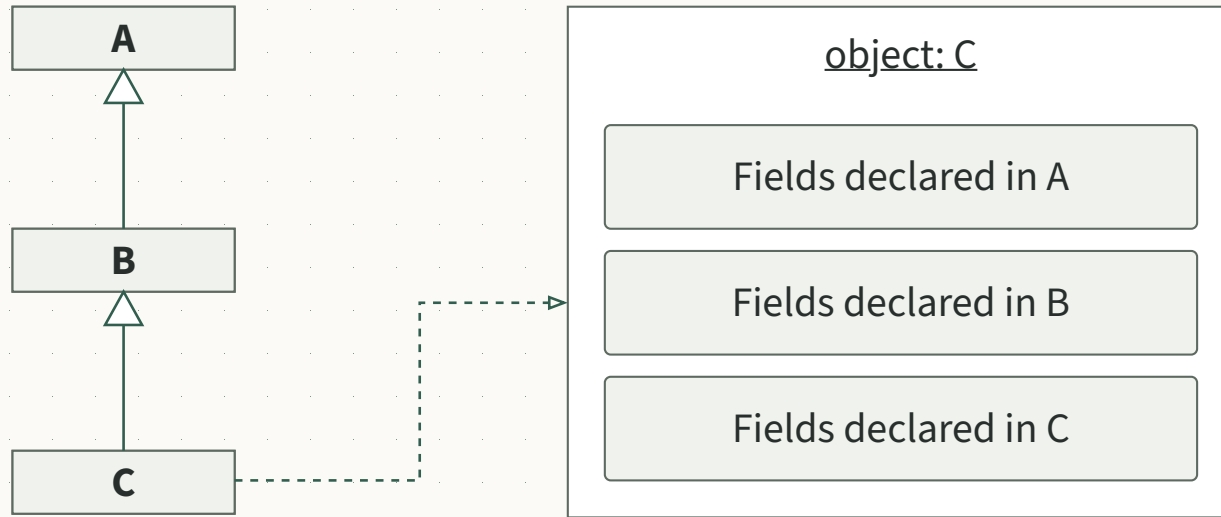
Java

```
1  public class Circle
2      extends Shape {
3      public double radius;
4
5      public Circle(double x,
6                     double y,
7                     double radius) {
8          setX(x);
9          setY(y);
10         this.radius = radius;
11     }
12 }
```

Java

3. Object Instantiation

One Object, All Instance Fields



One object: all instance fields, including private superclass state.

Methods belong to classes; their code is not copied into each object.

Constructor Chaining

Call super()



Initialize



First allocate all fields with default values; then run the constructor chain.

At each class: field initializers, then constructor body after super().

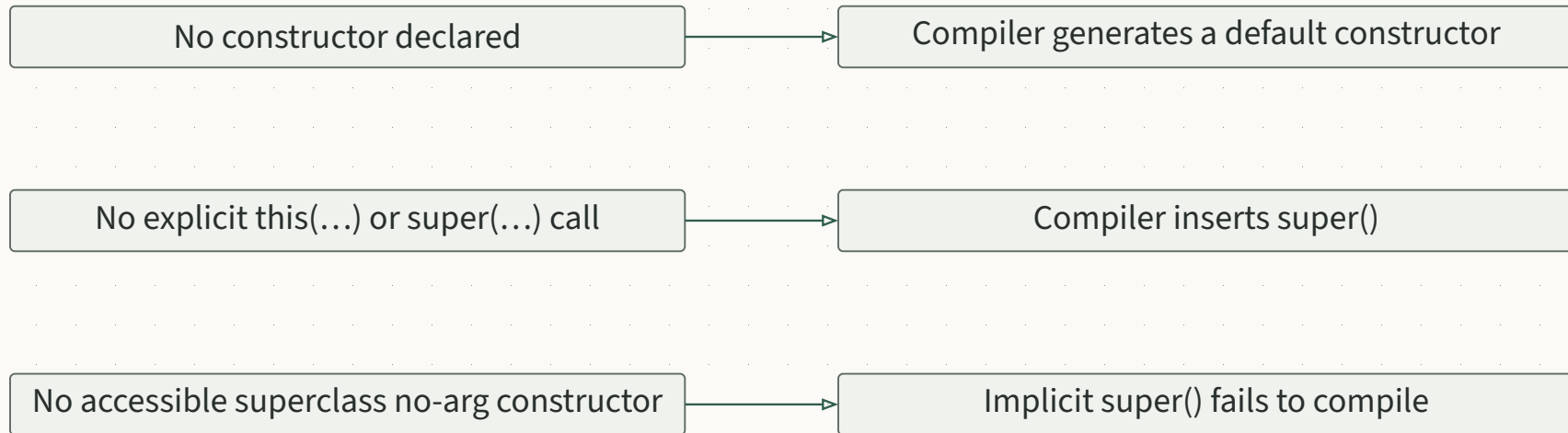
Calling the Superclass Constructor

Put `super( ... )` first in the constructors used in this lecture.

```
1  public class A {  
2      double a; // Initially 0.0  
3  }  
4  public class B extends A {  
5      double b;  
6  
7      public B(double b) {  
8          super();  
9          this.b = b;  
10     }  
11 }
```

Java

Calling the Superclass Constructor

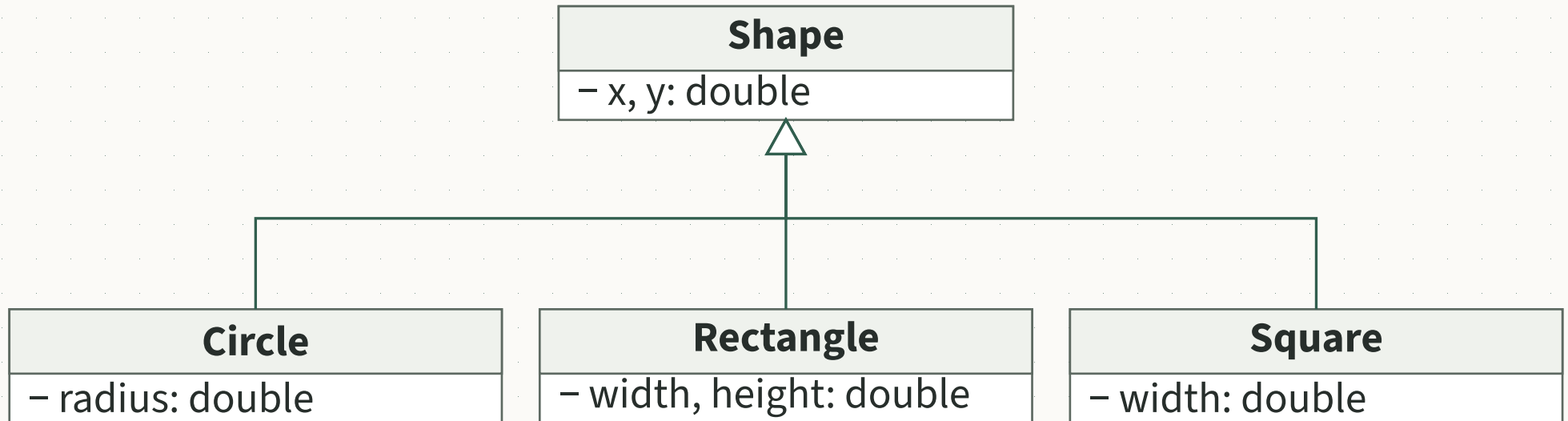


Task: Initialize Superclass State



Task 5

Add a Shape constructor for x and y. Call it from Circle instead of using setters.



Task: Initialize Superclass State

```
1 public class Shape {
2     private double x, y;
3
4     public Shape(double x,
5                 double y) {
6         this.x = x;
7         this.y = y;
8     }
9 }
```

Java

```
1 public class Circle
2     extends Shape {
3     public double radius;
4
5     public Circle(double x,
6                 double y,
7                 double radius) {
8         super(x, y);
9         this.radius = radius;
10    }
11 }
```

Java

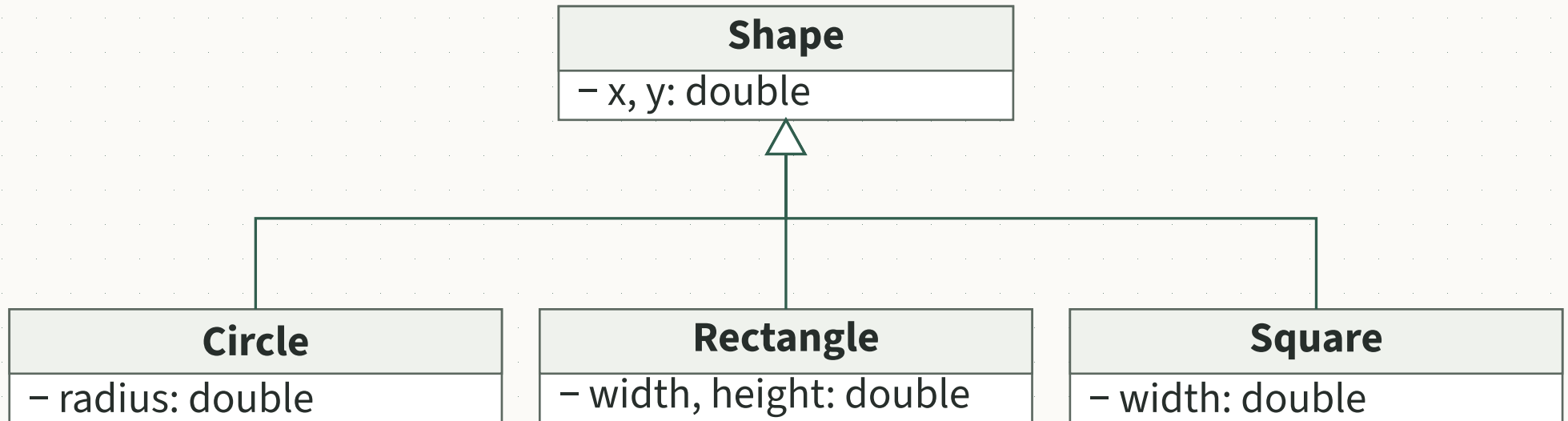
`super(x, y)` selects the matching constructor—not a default constructor.

Task: Protect the Shape Fields



Task 6

Make all fields private. Add constructors and only the getters/setters you need.



Task: Protect the Shape Fields

```
1  public class Circle extends Shape {  
2      private double radius;  
3  
4      public Circle(double x, double y, double radius) {  
5          super(x, y);  
6          this.radius = radius;  
7      }  
8      public double getRadius() {  
9          return radius;  
10     }  
11     public void setRadius(double radius) {  
12         this.radius = radius;  
13     }  
14 }
```

Java

4. Referencing via Base Class

Person and Pilot, with Constructors

```
1 public class Person {
2     String name;
3
4     public Person(String name) {
5         this.name = name;
6     }
7 }
```

Java

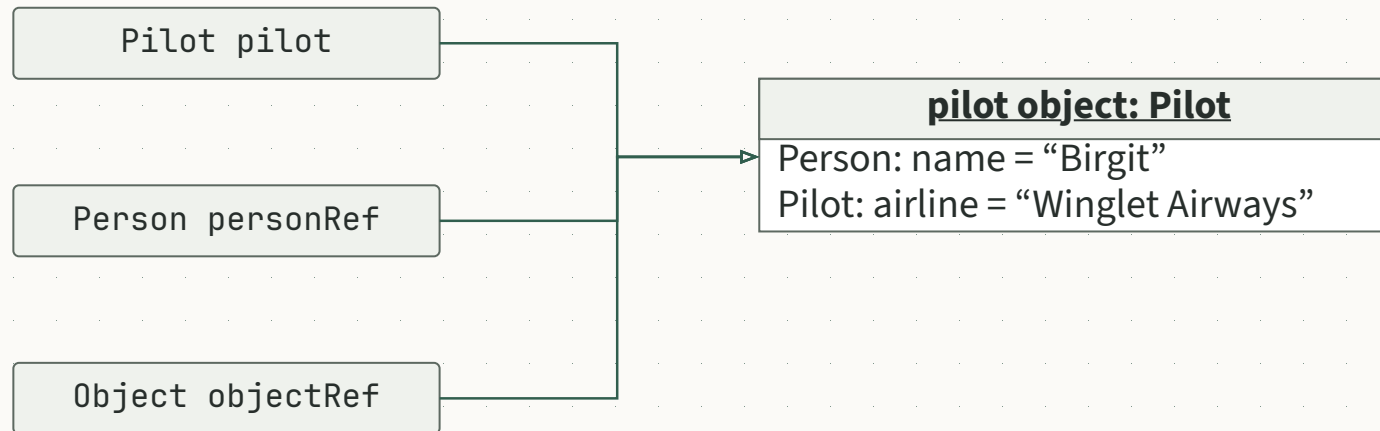
```
1 public class Pilot
2     extends Person {
3     String airline;
4
5     public Pilot(String name,
6         String airline) {
7         super(name);
8         this.airline = airline;
9     }
10 }
```

Java

One Object, Several Reference Types

```
1 Pilot pilot = new Pilot("Birgit", "Winglet Airways");  
2 Person personRef = pilot;  
3 Object objectRef = pilot;
```

Java



Three references · one object · runtime type stays Pilot

Reference Type Controls Access

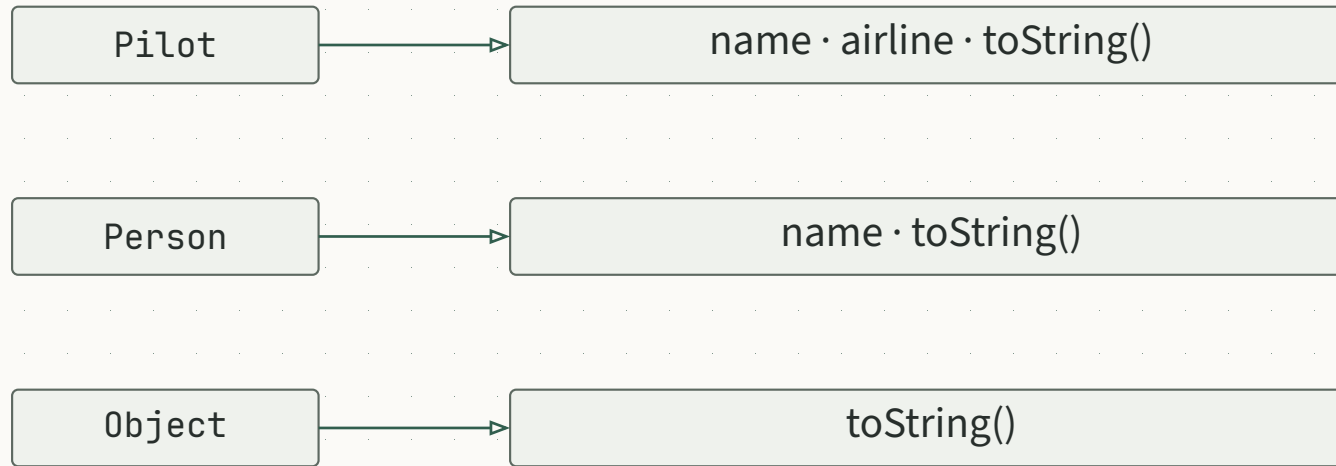


Which accesses compile? Assume all classes are in the same package.

```
1 Pilot pilot = new Pilot("Birgit", "Winglet Airways");  
2 Person personRef = pilot;  
3 Object objectRef = pilot;  
4  
5 System.out.println(personRef.name);  
6 System.out.println(personRef.airline);  
7 System.out.println(objectRef.name);  
8 System.out.println(objectRef.airline);
```

Java

Reference Type Controls Access

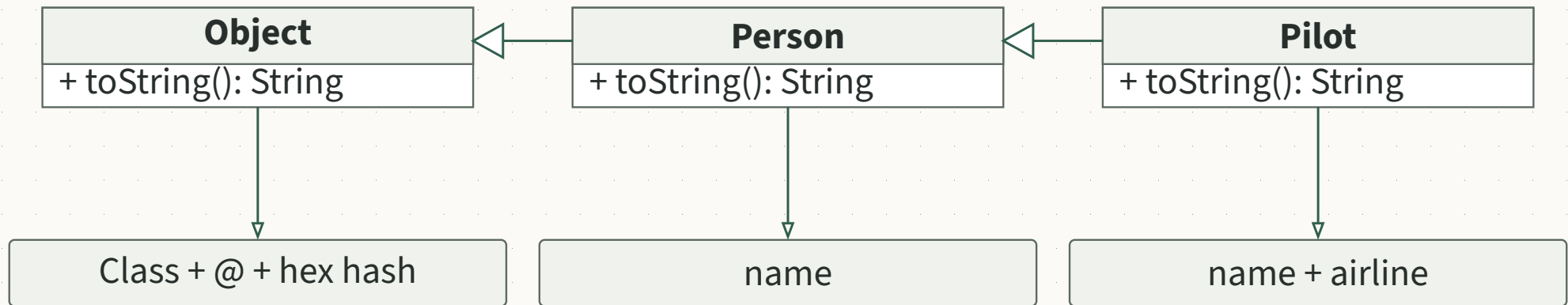


Reference type determines available members; access rules still apply.

Of the four accesses, only `personRef.name` compiles.

5. Overriding

A New Implementation of toString()



Same signature · subclass supplies a new implementation

Person Overrides toString()

```
1  public class Person {  
2      private String name;  
3      public Person(String name) {  
4          this.name = name;  
5      }  
6      public String getName() {  
7          return name;  
8      }  
9      @Override  
10     public String toString() {  
11         return name;  
12     }  
13 }
```

Java

Pilot Overrides toString()

```
1  public class Pilot extends Person {  
2      private String airline;  
3      public Pilot(String name, String airline) {  
4          super(name);  
5          this.airline = airline;  
6      }  
7      @Override  
8      public String toString() {  
9          return String.format("%s (%s)", getName(), airline);  
10     }  
11 }
```

Java

Which Implementation Runs?



What will this print?

```
1 Person person = new Person("Birgit Janssen");  
2 System.out.println(person);  
3 System.out.println(person.toString());  
4  
5 Pilot pilot = new Pilot("Jan Birgerson", "Winglet Airways");  
6 System.out.println(pilot.toString());
```

Java

Which Implementation Runs?



What will this print?

```
1 Person person = new Person("Birgit Janssen");  
2 System.out.println(person);  
3 System.out.println(person.toString());  
4  
5 Pilot pilot = new Pilot("Jan Birgerson", "Winglet Airways");  
6 System.out.println(pilot.toString());
```

Java

```
1 Birgit Janssen  
2 Birgit Janssen  
3 Jan Birgerson (Winglet Airways)
```

text

Runtime Type Selects the Method

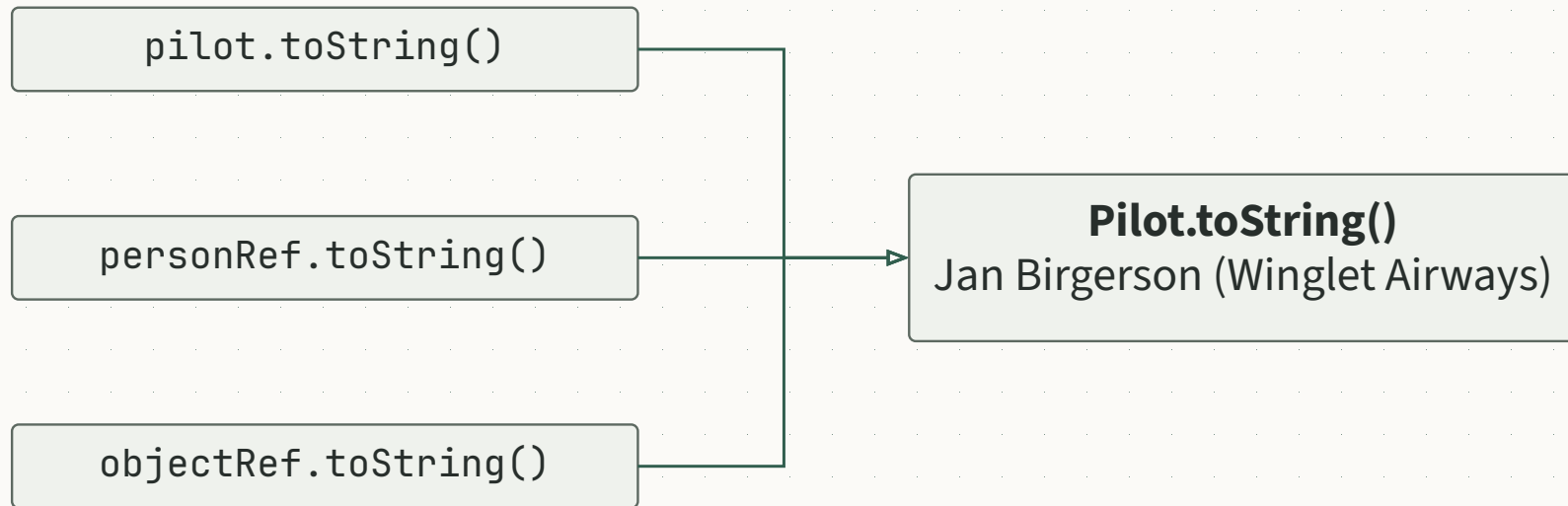


Does the reference type change the output?

```
1 Pilot pilot = new Pilot("Jan Birgerson", "Winglet Airways");  
2 Object objectRef = pilot;  
3 Person personRef = pilot;  
4 System.out.println(objectRef.toString());  
5 System.out.println(personRef.toString());
```

Java

Runtime Type Selects the Method

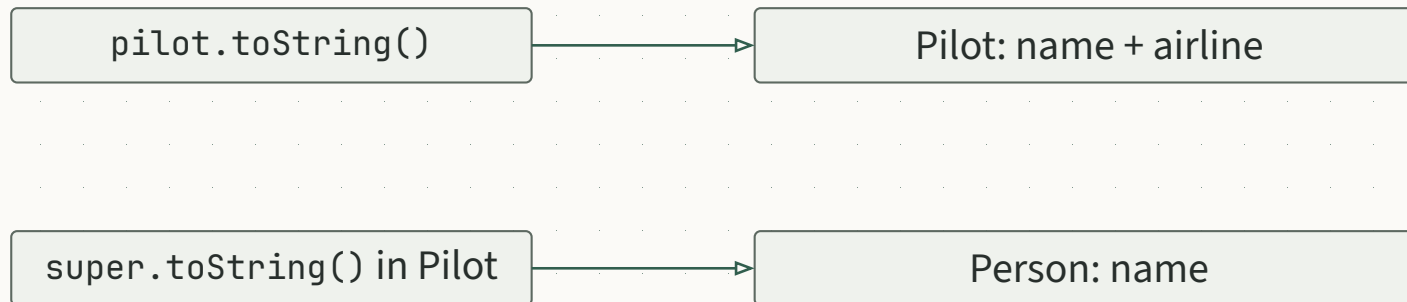


Runtime object type selects the overridden instance method.

Calling the Superclass Implementation

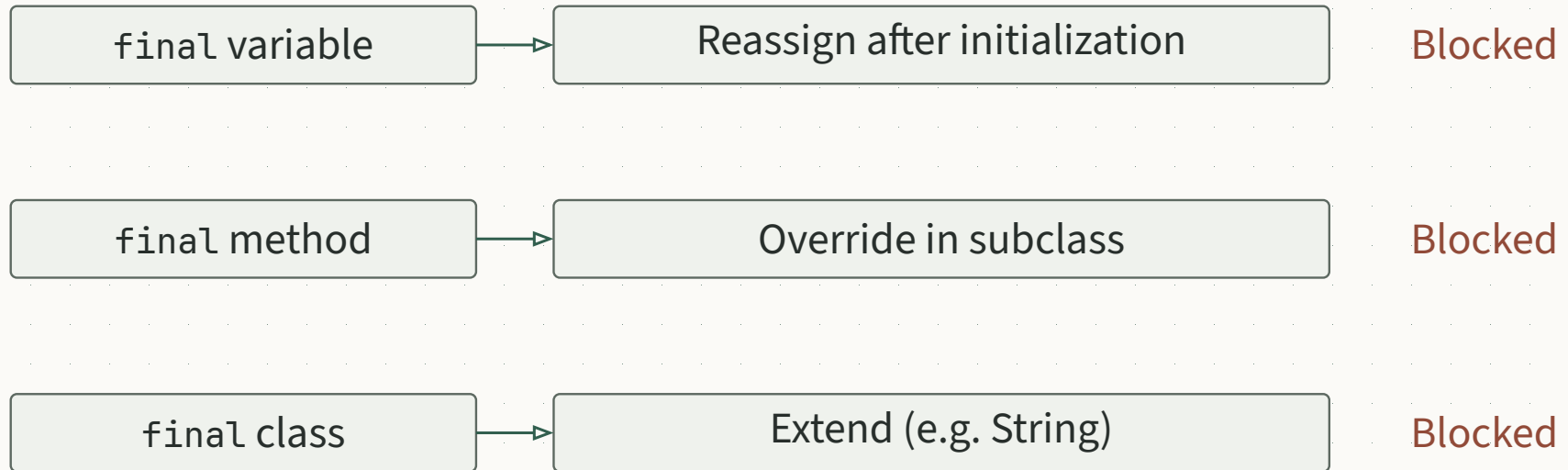
Inside Pilot:

```
1 public String toStringOfSuperClass() {  
2     return super.toString();  
3 }
```

Java

`super` selects the superclass implementation on the same object.

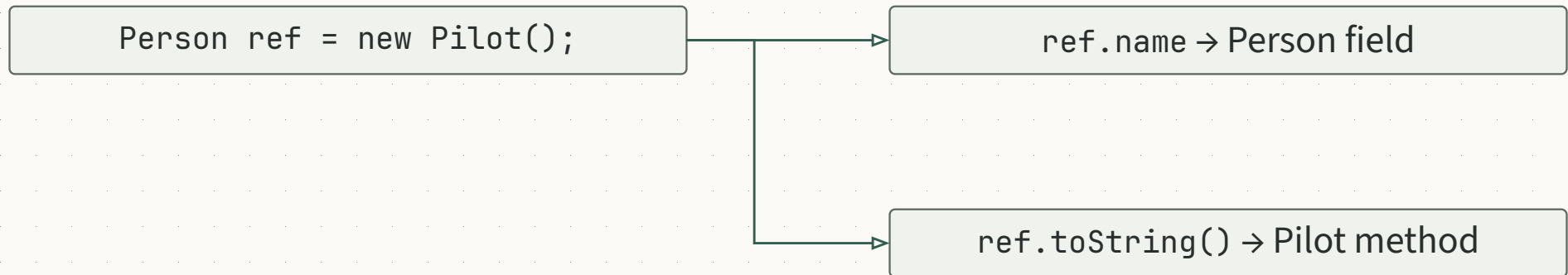
What Does final Prevent?



Final reference: no reassignment. The object may still change.

Fields Are Not Overridden

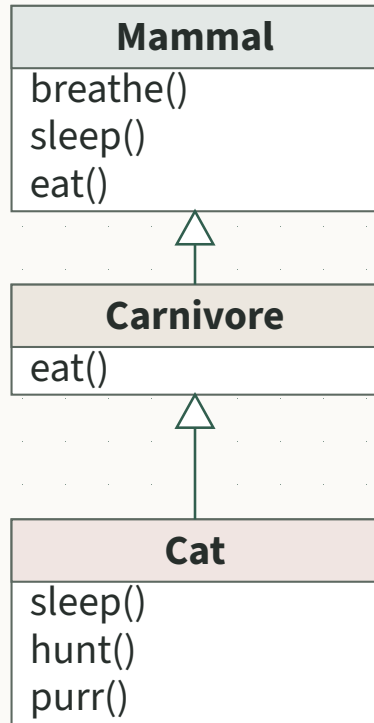
Suppose both `Person` and `Pilot` declare a public field named `name`.



Fields are hidden, not overridden. Avoid duplicate field names.

Static methods are also selected by compile-time type, not runtime dispatch.

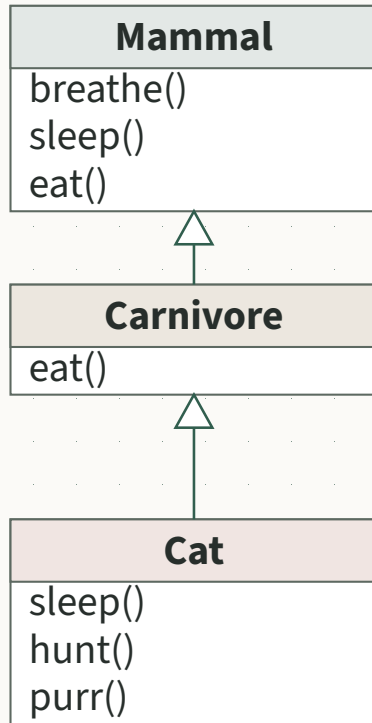
Exercise: Jason the Cat



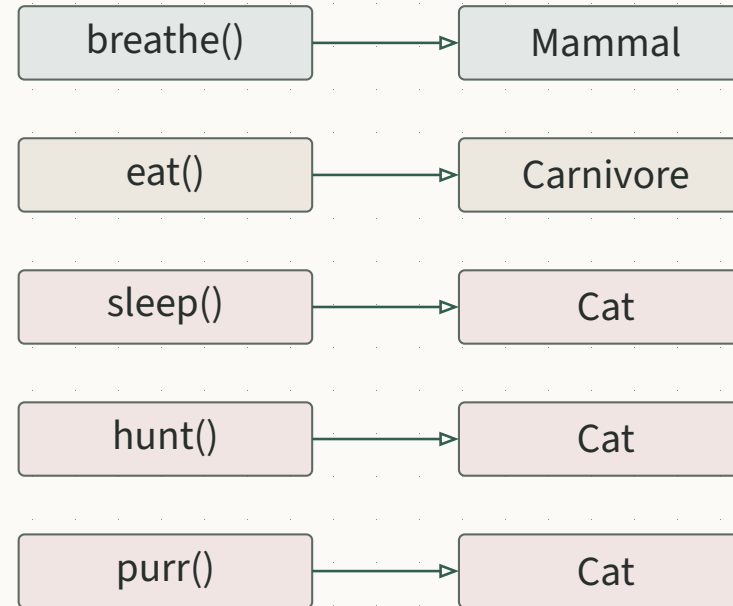
jason: Cat

Which methods?
Which defining class?

Exercise: Jason the Cat



jason: Cat



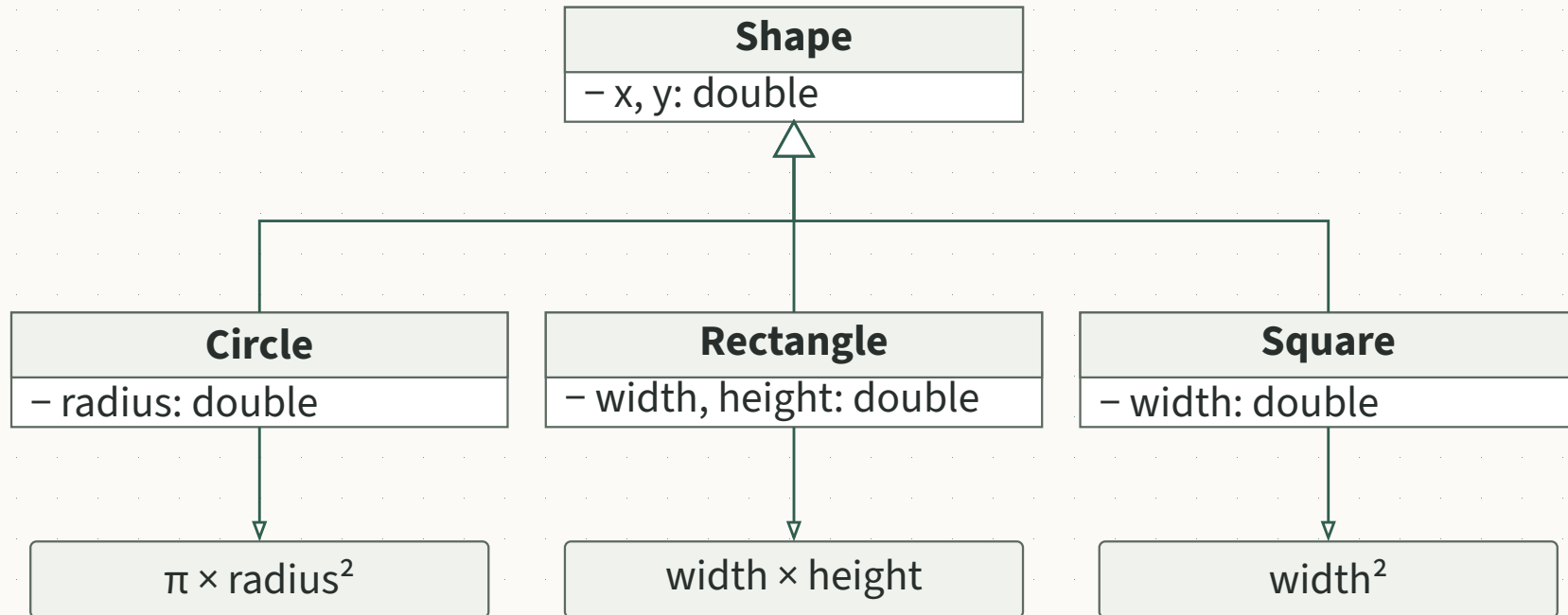
Task: Sum the Areas



Task 7

Add `getArea()` to the shape classes. Store one `Circle`, `Rectangle`, and `Square` in an `ArrayList<Shape>` and sum their areas.

Task: Sum the Areas



Same call: `getArea()` · implementation depends on the object

Task: Sum the Areas

```
1  // In Circle:
2  @Override
3  public double getArea() {
4      return Math.PI * radius * radius;
5  }
6  // In Rectangle:
7  @Override
8  public double getArea() {
9      return width * height;
10 }
11 // In Square:
12 @Override
13 public double getArea() {
14     return width * width;
15 }
```

Java

Task: Sum the Areas

For now, add this placeholder to Shape; each concrete shape overrides it.

```
1 public double getArea() {  
2     return 0.0;  
3 }
```

Java

A later lecture replaces the placeholder with an **abstract method**.

Task: Sum the Areas

```
1  // Import java.util.ArrayList; code inside main():
2  ArrayList<Shape> shapes = new ArrayList<>();
3  shapes.add(new Circle(2.0, 3.0, 1.0));
4  shapes.add(new Rectangle(-1.0, 0.0, 3.5, 4.0));
5  shapes.add(new Square(0.0, 0.0, 2.5));
6
7  double sumArea = 0.0;
8  for (Shape shape : shapes) {
9      sumArea += shape.getArea();
10 }
11 System.out.println("Overall area = " + sumArea);
```

Java

6. Polymorphism

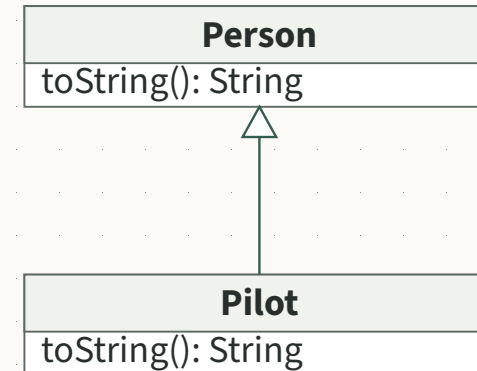
Same Name, Different Implementations

Overloading · compiler selects by arguments

MathFunctions
max(int, int): int max(int, int, int): int max(double, double): double max(double[]): double

Same name · different parameter lists

Overriding · runtime selects by object



Same signature · inheritance hierarchy

Overloading cannot differ only by return type.

7. Comparing Objects

Identity: The == Operator

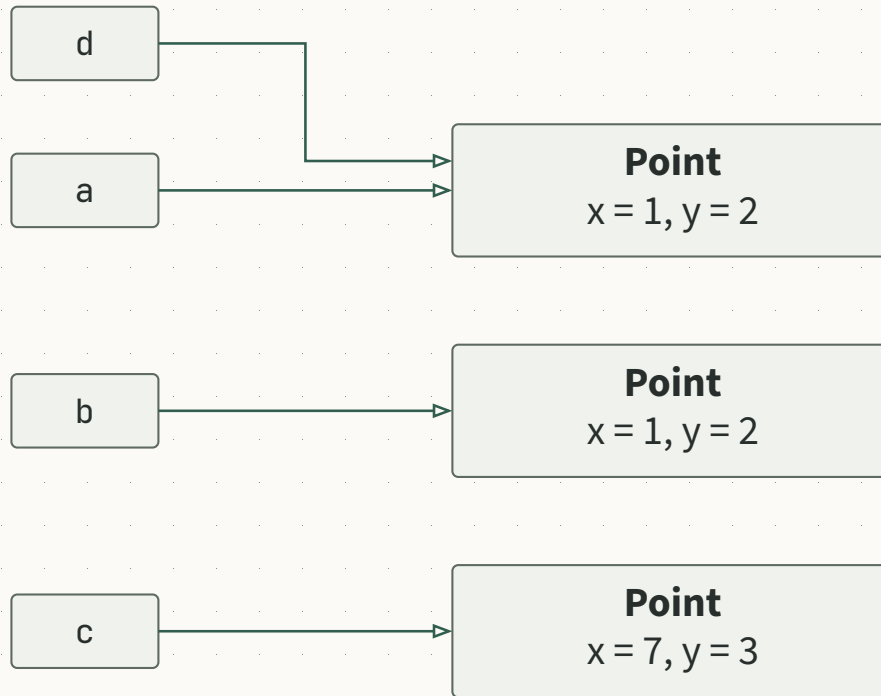


What does each comparison print?

```
1 Point a = new Point(1, 2);
2 Point b = new Point(1, 2);
3 Point c = new Point(7, 3);
4 Point d = a;
5
6 System.out.println(a == b);
7 System.out.println(a == c);
8 System.out.println(a == d);
```

Java

Identity: The == Operator



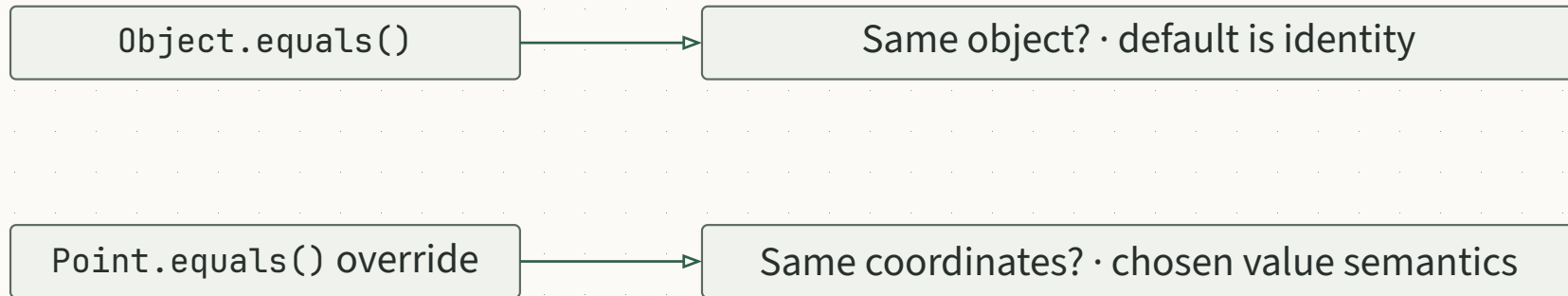
`a == d` → true

`a == b` → false

`a == c` → false

`==` compares references, not field values.

Value Equality: equals()

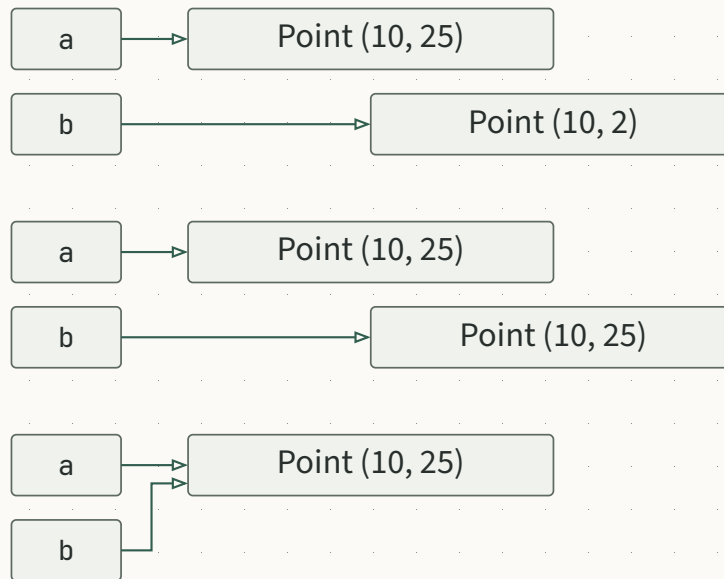


Equal objects must have equal hash codes: override `hashCode()` too.

Use your IDE's Generate menu to create `equals()` and `hashCode()` together.

Exercise: Identity or Equality?

References

`a == b``a.equals(b)`

?

?

?

?

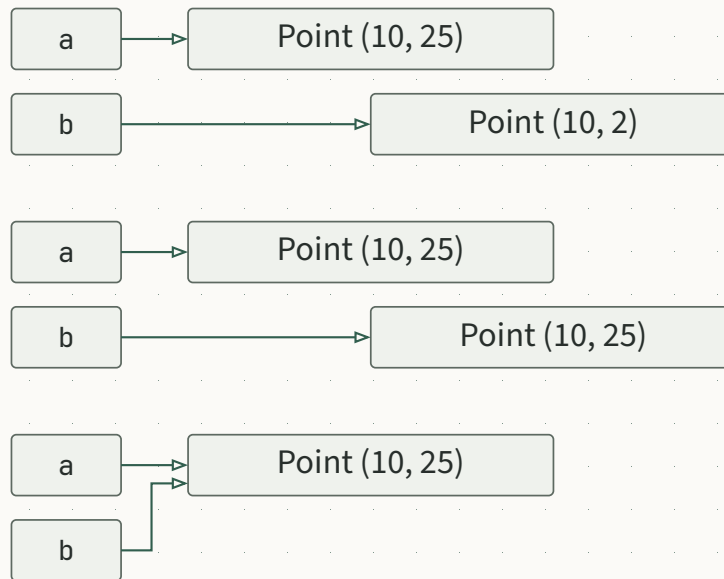
?

?

Assume `Point` overrides `equals()` to compare `x` and `y`.

Exercise: Identity or Equality?

References

`a == b``a.equals(b)`

false

false

false

true

true

true

Assume Point overrides equals() to compare x and y.

8. License Notice

Attribution

- Shared under CC BY-NC-SA 4.0 and the applicable public license.
- Based on teaching material by Prof. Dr. Marc Hensel.
- Text, diagrams, and layout adapted. Original material supplied privately; no public source link is available.